

Lab 10

Programing with Python Lab

BCACC393

AI Based Lab Assignment Questions

Part A — GUI-Based AI Applications

Q1. AI-Based Tic-Tac-Toe using GUI

Develop a Python GUI-based Tic-Tac-Toe game in which a human player competes against an AI agent. The AI should select its moves intelligently using the **Minimax algorithm**. The GUI should display the game board, player turns, and the final result.

Q2. GUI-Based 8-Puzzle Solver

Develop a Python GUI application for solving the **8-puzzle problem**. Allow the user to enter or generate an initial puzzle configuration and display the sequence of moves required to reach the goal state. Implement **Breadth-First Search (BFS)** and **A* search** and provide an option to compare their solutions.

Part B — Blind (Uninformed) Search

Q3. Breadth-First Search for Route Finding

Implement **Breadth-First Search (BFS)** in Python to find a path between two nodes in a given graph. The program should accept the graph, starting node, and goal node as input and display the path found along with the number of nodes explored.

Q4. Depth-First Search for Maze Solving

Develop a Python program that uses **Depth-First Search (DFS)** to solve a maze represented as a two-dimensional matrix. The program should identify a path from the starting cell to the destination cell and display the resulting path.

Part C — Heuristic Search

Q5. A* Search for the 8-Puzzle Problem

Implement the **A* search algorithm** in Python to solve the 8-puzzle problem. Use the **Manhattan distance** as the heuristic function. Display the initial state, intermediate states, final state, total path cost, and number of states explored.

Q6. Greedy Best-First Search for Route Planning

Develop a Python program implementing **Greedy Best-First Search** to find a route between cities represented as nodes in a graph. Use the estimated distance to the destination as the heuristic value. Compare the resulting path with the path obtained using BFS and discuss the differences in terms of path length and nodes explored.

Part D — Knowledge Representation

Q7. Knowledge Representation using Propositional Logic

Develop a Python program to represent a simple **knowledge base using propositional logic**. Consider a domain such as classroom activities. Store facts and rules in the knowledge base and implement a mechanism to determine whether a given proposition can be inferred from the available knowledge.

Q8. Knowledge Representation using a Semantic Network

Create a Python program that represents knowledge about a selected domain such as **vehicles** using a semantic network. Represent relationships such as **IS-A**, **HAS-A**, and **CAN**. The program should accept a query and determine relevant relationships from the knowledge representation.

Part E — PEAS and Intelligent Agents

Q9. Python Program to Generate PEAS Description

Develop a Python program that generates the **PEAS (Performance Measure, Environment, Actuators, Sensors)** description for a given AI task. The program should accept an AI application such as **vacuum-cleaner agent** and display its corresponding PEAS components in a structured format.

Part F — Integrated AI Problem

Q10. Intelligent Navigation Agent using Search and Heuristics

Develop a Python-based intelligent navigation agent that finds an optimal path through a grid environment. Implement **BFS, DFS, Greedy Best-First Search, and A***. Display the search process graphically using a GUI, compare the algorithms based on **path cost, number of nodes explored, and execution time**, and identify which algorithm performs best for the given environment. Also provide the **PEAS description** of the navigation agent.